

License Hunter: How It Actually Gets the License

A practical explanation of how an agent can turn a natural-language request into a real, verifiable purchase - not an invented license or a generic recommendation.

The question

"How can one get the license and stuff? How will it know the license is available? The user saying this and that is just text - if it is not feasible, how will it work?"

The short answer

The user's text is only the request. It is not evidence that a license exists. The actual marketplace or rights-holder must be the authority. License Hunter only recommends and purchases licenses when a connected provider returns a real asset ID, current license terms, current price, and a successful purchase response.

The workflow

1. Convert the request into requirements

The user says: "Find music for a worldwide paid YouTube ad, perpetual use, under \$50, no attribution." The system converts that into structured requirements:

```
{"asset_type": "music", "commercial_use": true, "paid_ads": true, "territory": "worldwide",  
"duration": "perpetual", "attribution": false, "budget": 50}
```

The important fields are commercial use, paid advertising, territory, duration, number of projects, modification rights, redistribution rights, exclusivity, attribution, and AI-training restrictions.

2. Search real provider catalogs

The agent calls connected providers, rather than inventing search results. Each provider connector returns a normalized asset record with a real provider ID, price, available license types, and source URL.

```
{"asset_id": "track_88421", "title": "Future Motion", "price": 29,  
"available_license_types": ["commercial", "advertising"], "license_url": "provider.com/license/track_88421"}
```

3. Ask the provider what license is actually available

For every candidate, License Hunter calls the provider's license-information endpoint. The response should identify the exact license type, price, territories, duration, paid-advertising permission, and attribution requirement.

If the provider does not return this information, the agent must mark the candidate as "needs review" or reject it. It should never infer permission from a title or a vague description.

4. Match the license against the requirements

Use deterministic rules for the final decision. The language model can extract terms from messy text, but a rules engine decides whether the terms satisfy the request.

Example: a \$9 personal-use license is rejected when the user requires paid advertising, even if the asset itself looks perfect.

5. Re-check immediately before purchase

Availability and price can change. The purchase sequence is: search -> inspect license options -> user approves asset and budget -> re-check price and availability -> purchase -> download receipt and asset.

The asset is not considered secured until the provider returns a successful purchase response with a purchase ID, license type, amount paid, license document URL, and download URL.

6. Store proof in a rights vault

The final deliverable is a rights package, not just an MP3 or image:

```
project-rights-pack/  
- asset file  
- purchase receipt  
- license certificate  
- license terms snapshot  
- attribution.txt  
- rights-summary.json
```

The terms snapshot is important because a provider can change its website terms later. The customer needs proof of what they purchased at the time.

How the agent can purchase

Option A: real provider API

This is the best option. Integrate with a provider that allows your application to search, license, and download assets. Adobe Stock, Shutterstock, and MotionElements all document API-based licensing capabilities. For references, see the Adobe Stock licensing API, Shutterstock API reference, and MotionElements API.

Option B: checkout handoff

If a provider allows search but not programmatic checkout, the agent returns the exact asset and purchase link. The user completes payment, uploads the receipt, and License Hunter verifies and stores the license. This is less autonomous but still completely feasible.

Option C: your own licensed catalog

For a hackathon, this may be the smartest route. Partner with 10-20 independent musicians, photographers, designers, or footage creators. Each contributor uploads the asset, price, permitted uses, territories, duration, attribution requirement, and prohibited uses. Your agent owns the checkout and issues the license instantly.

The realistic hackathon MVP

Do not support music, photos, fonts, video, datasets, and software all at once. Build License Hunter for commercial music in video ads.

A good demo:

1. User asks for music for a paid global ad.
2. Agent searches a real catalog.
3. It shows three tracks and their exact license terms.
4. It rejects one because paid advertising is prohibited.
5. User approves one.
6. Agent pays.
7. Provider returns the asset and license certificate.
8. Agent produces a rights pack.

The core rule

"The agent may interpret the request, but only the licensor can prove that the license exists."

Do not claim that the system guarantees legal safety. Position it as procurement assistance that matches assets against explicit license terms, records the transaction, and flags conflicts for human review.